Ski Resort - Project Criteria 1
Project Overview 2
Stakeholders: 2
( )Core Features 2
Customer Membership System 2
Possible Extensions 3
Equipment Rental System 3
Object-Oriented Programming Features 3
Ski Slope Management 3
Requirements 3
OOP Concepts 3
/\ Accommodation Search System 4
Customers should be able to: 4
Additional Logic 4
Reservation & Availability Systems 4
Accommodation Occupancy Simulation 5
Parking Reservation System 5
Additional Features 5
[:] Resort Packages & Services 5
Modular Resort Plans 6
Example Plans 6
Package Benefits 6
Modular System 6
£-Business & Pricing Logic 6
Resort Pricing System 6
Gondola Monitoring System 7
[] Database Requirements 7
Relational Database 7
Possible Tables include: 7

Relationships.....7
@# Searching, Filtering & Sorting.....7
Search Tools.....8
Examples.....8
£ Payment System.....8
Payment Validation.....8
Requirements.....8
Possible Extension.....8
User Interface.....8
Customer Features.....9
Staff Features.....9
Research & Evidence/ analysis.....9
Research Section.....9
Existing ski resort systems.....9
Suggested Research Areas (from el gpt).....9
Take into consideration existing systems.....10

Key:

Red = optional

Blue = necessary

Ski Resort - Project Criteria:

Project Overview

I intend to design and develop a comprehensive ski resort management system for a winter resort business. The system will manage customers, accommodation, memberships, reservations, ski facilities, parking, and resort services through an interactive user interface.

Stakeholders:

The main group to benefit from the new system will be adults intending to plan a holiday at my resort. They will be able to utilize the system to easily rent skiing equipment, reserve services such as the ski slopes themselves, manage their purchased perks, and/or book a room at the resorts many hotels, decreasing or increasing in price depending on distance from the resort, room size, and room attributes. This system improves operational efficiency while offering customers a straightforward process for planning and managing their stay.

Staff at the resort will also benefit from the new system, as the system will cause reduction in staffs' workload, and therefore, increasing efficiency with better time allocation on tasks which may need more resources to complete, such as inventory, and management of systems which need constant updating to function adequately, such as ski slope traffic, and equipment renting

Time/ Hardware constraints:

My project is constrained by the available development time, due to variables such as other a-level subjects, as well as exams. The project will be developed using Python on a standard computer, limiting complexity of graphics and process-intensive features. As I am still developing my programming skills on more advanced concepts the project will focus more on core functionality such as managing customers, ski slope passes, accommodation availability, package plans, etc rather than implementing advanced

features such as online payments, simulation of car park space management (if time is scarce) or live weather integration.

( )Core Features:

Customer Membership System

- Customers can register/sign up for a membership account
    - Allowing for the customers to save their data, means that they dont have to keep reapplying for certain benefits or plans when booking again or renewing/changing their plan
- Why each feature is necessary

Possible Extensions

- Tiered memberships (Silver, Gold, Platinum)
    - Provides different benefits such as- reservations can be booked earlier, access to better services – this shows OOP and classes being used(different membership levels = different privileges, as well as possible use of database to check validity of member and id number. Utilise hashing to conceal private data
    - Benefits can also include accessing certain services before public opening, and after standard closing times, such as the gym, ski resort, or buffet/bar
- Automatic discount calculations
- Why each feature is necessary
- Members earn loyalty points from purchases and bookings.
    - Increases likelihood of customer loyalty to the branch
- Points can be redeemed for discounts on resort services.
- Different membership benefits may apply depending on customer activity or package type.
    - Example of OOP
-

Equipment Rental System

- Customers can rent skiing equipment such as:
    - Ski poles
    - Skis
    - Helmets
    - Utilising a clock system, with constant checks every iteration; utilises iterative techniques such as while loops
    - As well as for loops utilising the previous clock system for the rental function (using time.sleep, I am able to simulate time in a for loops, so i would be able to make durations, which can be used to simulate duration of occupancy to a customer
- Rental availability updates dynamically.
    - Real time updates reduce risk of over renting equipment; renting equipment which is not available

(Optional) determining the price variations:

- Rental pricing may vary based on duration of usage, equipment difficulty:
    - In terms of customer safety, increasing prices on more specialist equipment may discourage its usage to customers who may not be familiar with it, ensuring people who are familiar and comfortable with using specialist equipment are able to use it (improves the traffic of the system passively

Object-Oriented Programming Features:

Ski Slope Management

Use OOP/polymorphism to represent different ski slopes.

Requirements

- Different slopes have different:
    - Difficulty levels/steepness
    - Capacities - maxnum

    - Prices – could increase based on traffic on certain slopes
- Customers can view slope information such as business and availability, as well as difficulty/steepness and obviously slope price
- Customers can see how busy a slope is in real time – simulation utilising iteration and while loops; also uses randint
- Why each feature is necessary

OOP Concepts

- Classes- slopes
- Inheritance- slopes, accommodation
- Encapsulation(data abstraction)- customer

/\ Accommodation Search System

Create an accommodation management system using OOP, polymorphism

Customers should be able to:

- Search accommodation using filters such as:
    - Number of beds
    - View availability
    - Shower/bathroom facilities
    - Distance from slopes
- Sort accommodation by:
    - Price
    - Distance
    - Rating/features
    - Why each feature is necessary

Accomodation should be sorted into four levels sorted by room size, number of

Additional considerations

- Resort-owned accommodation becomes cheaper the further it is from the main skii area.
- Discounts apply when staying within the Musa resort branch.

Reservation & Availability Systems:

Accommodation Occupancy Simulation - queues

- Simulate rooms becoming occupied or vacant.
- Staff and customers can view live room availability.
- The system outputs:
    - Available rooms (OOP,polymorphism,queues)– will allow staff and customers to view which rooms are available (use of queues and popping to leave only the available rooms)
    - Reserved rooms - will allow staff to see who has reserved the room and for how long and customers to view which rooms are reserved and for how long
    - Occupied rooms - will allow staff to see the occupants, and customers to see how long the room will be occupied(for online planning)

Parking Reservation System

- Customers can reserve parking spaces.
- Parking can be booked up to 3 days before a reservation begins.
- Parking spaces use colour-coded sections, as a passive wayfinding system, as well as a systematic way to record spot usage: - OOP, polymorphism
    - Red
    - Green
    - Blue
    - Yellow
    - Purple
    - Orange
    - Why each feature is necessary – this colour coded system adds structure to the system; customers may hopefully find it easier to find their cars; faster access speeds mean less usage time, allowing more customers to utilise the car park



An example of this type of system in use at Bluewater, used to improve wayfinding therefore

Additional Features

- Track occupied and available parking spaces
- Link parking reservations to customer bookings
    - If all data to do with a customer is stored in one place, it means problems to do with a customer can easily be indentified and resolved
- Why each feature is necessary

[:] Resort Packages & Services:

Modular Resort Plans

Customers can choose different resort packages.

Example Plans

- All Inclusive Plan
- Leisure Plan (other facilities such as gym/spa
- Nourriture (Food) Plan
- Why each feature is necessary

Package Benefits

Packages may include:

- Breakfast
- VIP lounge access – membership perks
- Spa access
- Bar access
- Equipment discounts
- Why each feature is necessary

Modular System

Plans are modular, meaning customers can customise or combine services.

Use of encapsulation for the customer

£-Business & Pricing Logic:

Resort Pricing System

- Prices vary depending on:
    - Distance from the skii slopes
    - Accommodation type
    - Membership status
    - Package selected
    - Why each feature is necessary

- CHECK YOUR PLAN AND ADD OR REDUCE MODULES TO YOUR PLAN - encapsulation

Gondola Monitoring System

- Live tracking of gondola usage/capacity - Customers can view how busy gondolas are in real time
- Simulation of live usage and gondola traffic – use of iteration
- Why each feature is necessary

[] Database Requirements:

Relational Database

The system should use multiple related database tables.

Possible Tables include:

- Customers - memberships
- Memberships
- Reservations
- Accommodation
- Slopes -
- Parking - red, green blue, orange purple
- Packages – norriture/food package, spa package etc.
- Payments
- Why each feature is necessary
- HOW TO IMPROVE ON THIS SECTION:
- Diagrammatic example, with use of 1st, 2nd , 3rd NF

Relationships

- One-to-many
- Many-to-many
- Foreign keys

@# Searching, Filtering & Sorting:

Search Tools

Users should be able to:

- Filter data based on attributes
- Sort results efficiently, depending on the size of the list being sorted, filtered etc, decides type of sorting/ searching algorithm will be used (binary and merge on big lists, linear and insertion on smaller ones)
- Why each feature is necessary

Examples

- Accommodation filters
- Slope difficulty filters
- Price sorting
- Availability sorting

£ Payment System:

Payment Validation/Sanitisation

Implement a payment system with card validation.

Requirements

- Use the Luhn Algorithm (checksum) to verify payment card numbers.
- Prevent invalid card details from being processed.

Possible Extension

- Use an API or test payment card dataset for validation/testing purposes.

User Interface:

Create an intuitive interface for both customers and staff.

Customer Features

- Booking:
    - Accommodation
    - Renting equipment
    - Reserving parking
    - plans/packages
- Viewing slope activity
- Managing memberships
- Why each feature is necessary

Staff Features

- Monitoring occupancy
- Viewing reservations
- Managing slope capacities
- Updating accommodation availability
- Why each feature is necessary

Research & Evidence/ analysis:

Research Section

Include evidence and research for:

Existing ski resort systems

Difficulty of slopes which can affect price for customer

- Reservation systems
- Membership systems
- Payment validation methods
- Database design
- User interface design principles
- Why each feature is necessary

Suggested Research Areas (from el gpt)

- Real-world ski resort booking systems
    - Ive used club med as an example
- Hotel management systems
- Queue/capacity monitoring systems
    - Looked at bluewaters passive car park system
- Card validation algorithms
    - Luhn algorthim

My project demonstrates:

- Object-Oriented Programming - polymorphism
- Algorithms
- Searching and sorting
- Relational databases
- Simulations – random and while loops(iteration)
- Real-time availability systems
- Validation

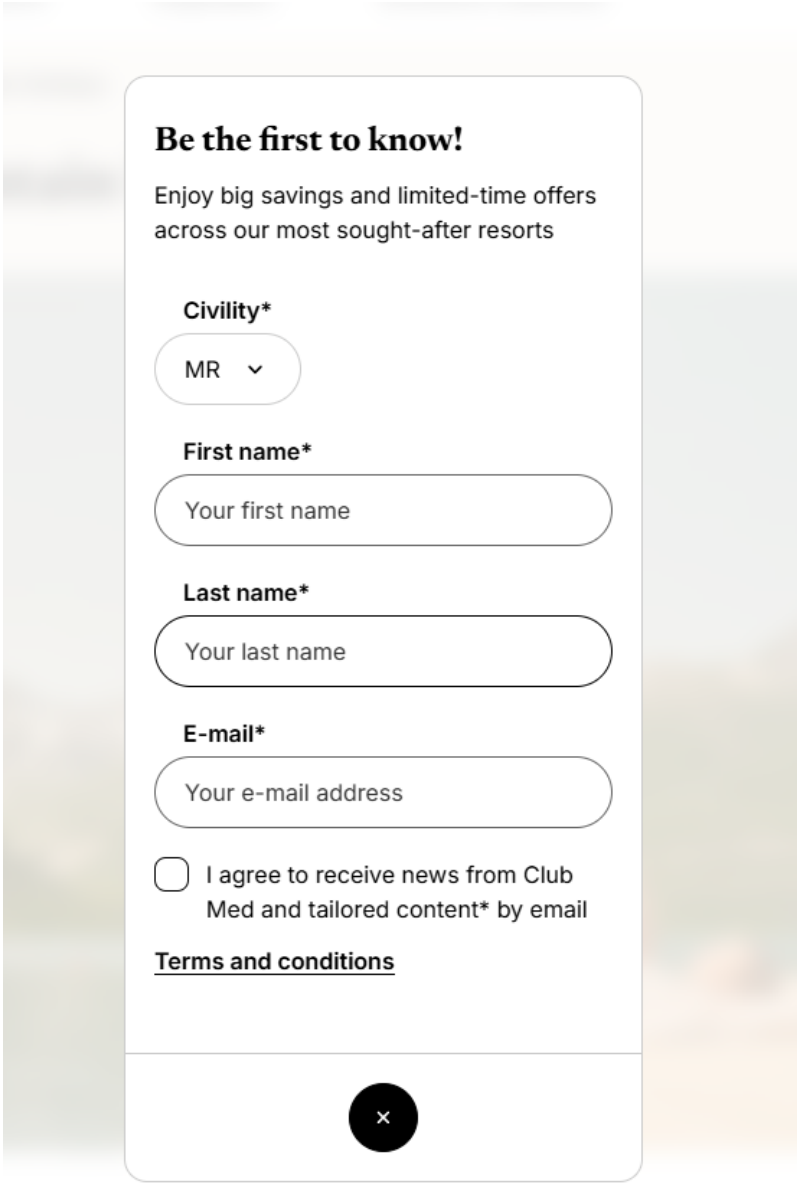
- Modular design
- Creation of a friendly UI
- Why each feature is necessary

Taking existing systems into consideration

Take into consideration real life statistics when simulating rental return systems and accommodation systems.

Resort in the alps example:

Membership/reward system, increasing likelihood of loyalty to the resort/branch



Filter system(merge sort):

You looking for?

France ✓ | Brazil | Japan | Dominican Republic

See all

Sun or Snow

Sun ✓ | Snow

Childcare by age

Please provide your child's birthdate and dates to get availabilities

0 to 2 years | 2 to 3 years | 4 to 10 years | 11 to 13 years | 14 to 17 years ✓





Opio en Provence
France
4.4/5



La Rosiere
France Summer
4.5/5

Establishing layout for the application:

Main classes:

1. Customer:
    - Booking:

Staff:

System traffic:

inventory

Accommodation:

SkiSlope:

Equipment:

Rental Package

Reservation

Payment System

membership

FEEDBACK:

1. **The six biggest improvements**

1. Define the problem more precisely

Your overview describes a comprehensive resort system, but it doesn't yet establish exactly what is wrong with the current way resort operations are managed.

Explain the specific problems your proposed system is intended to solve, such as difficulty checking availability across services, managing reservations, or keeping rental inventory up to date. Treat these as problems to investigate, not facts you've already proved.

2. Strengthen stakeholder analysis

You identify customers and staff, which is a good start. However, you need more detail about their different needs, difficulties, priorities and expectations.

Gather evidence through research, questionnaires or interviews where feasible. Then show how that evidence influences your requirements.

3. Make requirements specific and testable

Many of your requirements currently say things like "customers can rent equipment" or "staff can monitor occupancy."

Specify what the system must accept, process, store and display, including what should happen when an operation cannot be completed.

4. Improve research and evidence

Your document mentions Club Med, Bluewater's parking system and the Luhn algorithm. Build these into documented research findings instead of leaving them as brief notes.

Explain what you investigated, what you discovered, how reliable the information is, and what you will adopt or change for your own system.

5. Control the project scope

Your proposed system covers a large number of interconnected features. Decide which are essential, which are optional, and which you will exclude if development time is limited.

A smaller, reliable system with clear technical depth is preferable to a huge set of partially implemented features.

6. Add measurable success criteria

Define how you'll know the finished system has met its objectives. These criteria should follow from your identified problems and stakeholder needs, and should later be testable.

2. Improve your existing sections individually

A. Project overview and problem definition

What's missing: a clear explanation of the existing problem and why your proposed system is an appropriate solution.

Your current overview on page 3 describes the features and intended benefits, but doesn't establish a clear baseline against which the system can be judged.

Add a subsection called Problem definition. Explain:

How a customer might currently organise accommodation, equipment and other services.

What difficulties could occur if availability and bookings are handled separately.

What staff need to manage and update.

Why integrating these processes could improve the experience.

What your proposed software will and won't attempt to solve.

Avoid claiming that a real resort currently has these problems unless your research supports it.

B. Stakeholders

Your current Analysis describes customers and staff, but their requirements need to be developed further.

Stakeholder, Needs to investigate, Possible requirement

Customer, Simple booking, clear prices, accurate availability, Search for available accommodation using dates and preferences

Resort staff, Accurate reservations and inventory, View and update reservation and equipment records

Resort management, Consistent pricing and operational oversight, Review reservations and monitor resort capacity

System administrator, if included, Account management and access control, Manage authorised user accounts and permissions

For each stakeholder, explain why the need matters and how you obtained evidence for it.

If you can't access real resort staff or customers, be transparent. You can use published research and existing-system analysis, but don't present assumptions as interview findings.

C. Core features

Pages 4–12 contain most of your functional ideas, but some entries are still reminders to yourself, such as "why each feature is necessary."

Replace these reminders with a consistent structure:

Requirement: What must the system do?

Reason: Why is it necessary?

Evidence: What research or stakeholder need supports it?

Priority: Is it essential or an extension?

Success criterion: How will you test whether it works?

For example, take your equipment rental feature.

Example: Equipment rental analysis

An example of how to develop your existing idea into a more precise requirement.

Problem: Equipment must not be allocated to multiple customers for overlapping rental periods.

Functional requirements:

The customer can select an equipment type and rental period.

The system checks availability before confirming a rental.

The system records the rental period and the selected equipment.

The system prevents an unavailable item from being rented.

Staff can view active rentals and update equipment availability when items are returned.

Success criterion: Given an item that is already rented for a particular period, the system rejects a second rental that overlaps that period. A rental for a non-overlapping period is allowed when the item is otherwise available.

Evidence needed: Research how rental availability is handled in an existing booking or equipment management system, or document feedback from a relevant stakeholder.

Use the same method for accommodation, membership, parking, packages and staff management.

D. Research and existing systems

Your research section on pages 13–14 is currently more of a list of subjects to investigate than a record of findings.

I'd use a table like this to organise the research:

Research area, What to investigate, How it informs your solution

Existing ski resort booking system, Booking journey, available filters, packages and pricing, Customer interface and booking requirements

Hotel reservation systems, Date availability and overlapping bookings, Reservation validation

Parking wayfinding, Colour-coded parking areas and locating spaces, Parking interface and data representation

Membership schemes, Tiers, discounts and eligibility, Membership requirements and pricing

Sorting algorithms, Suitability and efficiency for the data involved, Search and sorting requirements

Luhn algorithm, How the checksum works and its limitations, Card-number validation, if implemented

For each area, record the actual source, what you learned and how it affected your design decisions.

Don't simply describe a website or algorithm. Explain what the research means for your project.

E. Scope, constraints and feasibility

Your constraints section on page 3 is a good starting point. You mention limited development time, Python, hardware limitations, sample data and potentially excluding advanced features.

Skiing resort (2).pdf

However, make these constraints actionable.

I'd divide your features into three groups:

Priority, Suggested features

Core, Customer accounts, accommodation search, booking and availability, relational data storage, staff reservation management

Additional, Equipment rental, membership discounts, packages, search and sorting

Optional extensions, Parking simulation, simulated gondola traffic, advanced live occupancy, payment-card validation

These are my suggested priorities, not priorities established in your document. Adjust them according to your stakeholder evidence and the technical objectives of your project.

Also explain feasibility:

Technical: Can you implement the feature with your current Python skills and chosen database?

Time: Can you implement and test it before your deadline?

Operational: Will customers and staff be able to use it effectively?

Data: Can you create realistic sample data to test the feature?

One point to reconsider is your proposed reduction in staff and operating expenses. Your Analysis currently suggests that the system will reduce staffing requirements and OPEX.

Skiing resort (2).pdf

Unless you have evidence to support this, frame the objective as reducing repetitive administrative work and improving efficiency, rather than promising that fewer staff will be needed.

F. Success criteria

This is one of the most useful additions I would make to your document.

Write a numbered list of clear, observable criteria that you can test later. For example:

ID, Success criterion, How to verify it

SC1, Customers can search accommodation using their chosen dates and preferences, Test matching and non-matching searches

SC2, The system prevents overlapping reservations for the same accommodation, Attempt conflicting bookings

SC3, Customers can view the total cost of their selected accommodation and package, Compare calculated totals against manually calculated expected values

SC4, Staff can view and update reservation records, Create, retrieve and modify test records

SC5, The system rejects invalid input without corrupting stored data, Use invalid and boundary test cases

SC6, Customers cannot access staff-only functions, Test access using accounts with different permissions

SC7, Equipment availability changes appropriately after a rental or return, Check inventory before and after each operation

Give each criterion a unique ID and use it consistently in your later design, implementation, testing and evaluation.

Only include criteria for features you genuinely intend to implement.

3. Separate Analysis from Design

Your current PDF also includes material on class layouts, merge sort and testing notes near the end.

Skiing resort (2).pdf

Some overlap between sections is normal, but make sure each part serves the correct purpose.

Analysis should establish..., Design should specify...

The need for a customer membership system, The membership classes, attributes and methods

The need to prevent double bookings, The algorithm used to detect overlapping dates

The need for a relational database, The tables, keys and relationships

The need for effective searching, The chosen searching and sorting algorithms

The need for accessible customer and staff interfaces, The actual wireframes, navigation and controls

The need to handle invalid input, The validation rules and error-handling logic

For example, your Analysis can explain why accommodation availability matters and define the requirement that bookings must not overlap. Your Design then explains precisely how the system will check dates and enforce that requirement.

Similarly, ideas such as using time.sleep(), while loops and randint to simulate activity are primarily implementation or design considerations. In the Analysis, focus on the need for the simulation and the information it must provide, then explain your technical approach in Design.

4. Your revised Analysis contents page

I'd reorganise your existing material into this structure:

1. Introduction and problem definition

Project purpose, existing problem, proposed solution and scope.

2. Stakeholder analysis

Customers, staff, management, their needs and supporting evidence.

3. Research and existing systems

Research findings, comparisons, source reliability and implications.

4. Functional requirements

Accounts, memberships, accommodation, reservations, equipment, packages, pricing, parking and staff operations.

5. Non-functional requirements

Usability, reliability, security, maintainability and performance where relevant.

6. Constraints and feasibility

Time, technical skills, hardware, data availability and feature prioritisation.

7. Success criteria

Numbered, measurable outcomes linked to requirements and later tests.

This is a suggested organisation, not a mandatory AQA template. Make sure you check the current AQA NEA guidance for the assessment requirements.

5. What I would do first

Don't rewrite all 18 pages immediately. That would take time away from development and testing.

I'd work through these four tasks first:

Write a clearer problem definition based on evidence.

Complete stakeholder needs and research findings.

Convert feature notes into specific, prioritised requirements.

Write numbered success criteria that can be tested.

Once these are done, much of your existing material can be retained and improved rather than discarded.

My overall assessment: you have enough feature ideas for a substantial project. The biggest gains will come from stronger evidence, clearer requirements, a realistic scope and measurable success criteria—not from adding even more features.

If you're working towards a higher mark, I'd start with your stakeholder analysis and success criteria, because they establish why the system is needed and how you will demonstrate that it solves the identified problem.

MORE EBIS:

REVISE OVER THE CULTURAL ETHICAL AND LEGAL CONCERNS

E.G.

AS CULTURE CHANGES WEBSITES SUCH AS THAT HANDLE CUSTOMER DATA NEED TO CORRECTLY ADDRESS THE CUSTOMERS TITLE, GENDER, AND OTHER FIELDS OF THEIR PERSONAL DATA THAT NEED OPTIONS TO BE WIDELY TAILORABLE

LEGAL CONCERNS IN HOW LONG WE KEEP CUSTOMER DATA FOR; EASY FIX IS REQUIRING CUSTOMERS TO GIVE NOTICE THAT THEIR DATA IS STILL BEING KEPT ON THE RESORTS SYSTEM;DATA RE-ENTRY ON PERSONAL DETAILS SUCH AS NAME DOB

State how many occupants can stay in the resort and its components, basing this on pre existing systems

Parking: 1000